Assignment 2

Name: Kartik Pawar

PRN: 2124UCSM1045

Dept: Cyber Security

LAB - 2

Experiment: Implementation of Monoalphabetic Substitution Cipher

Write a program to perform encryption and decryption using a Monoalphabetic Substitution Cipher, where each letter of the plaintext is replaced with a corresponding letter from a scrambled 26-letter key.

Theory

A Monoalphabetic Substitution Cipher is a symmetric-key encryption technique in which each letter of the standard alphabet is mapped to a unique letter of a scrambled (shuffled) 26-letter key alphabet. Unlike the Caesar cipher, which shifts every letter by the same fixed amount and therefore has only 25 possible keys, a substitution cipher key can be any permutation of the 26 letters, giving 26! (about 4 x 10^26) possible keys. This enormous key space makes brute-force attacks impractical. However, since the mapping between plaintext and ciphertext letters stays fixed throughout the message, the cipher does not hide the statistical structure of the underlying language. This makes it vulnerable to Frequency Analysis, where an attacker studies how often each letter appears in the ciphertext and compares it with known letter frequencies of the language to recover the key.

Algorithm

Encryption:

1. Define the standard 26-letter English alphabet (a-z).

2. Accept the plaintext and a 26-letter shuffled key string as input.

3. Validate that the key is exactly 26 alphabetic characters.

4. For each character in the plaintext:

   a. If it is a letter, find its index in the standard alphabet and fetch the character at the same index in the key.

   b. Preserve the original case (uppercase/lowercase).

   c. If it is not a letter, keep it unchanged.

5. Display the resulting ciphertext.

Decryption:

1. Take the ciphertext and the same key used for encryption.

2. For each character in the ciphertext:

   a. If it is a letter, find its index in the key and fetch the character at the same index in the standard alphabet.

   b. Preserve the original case.

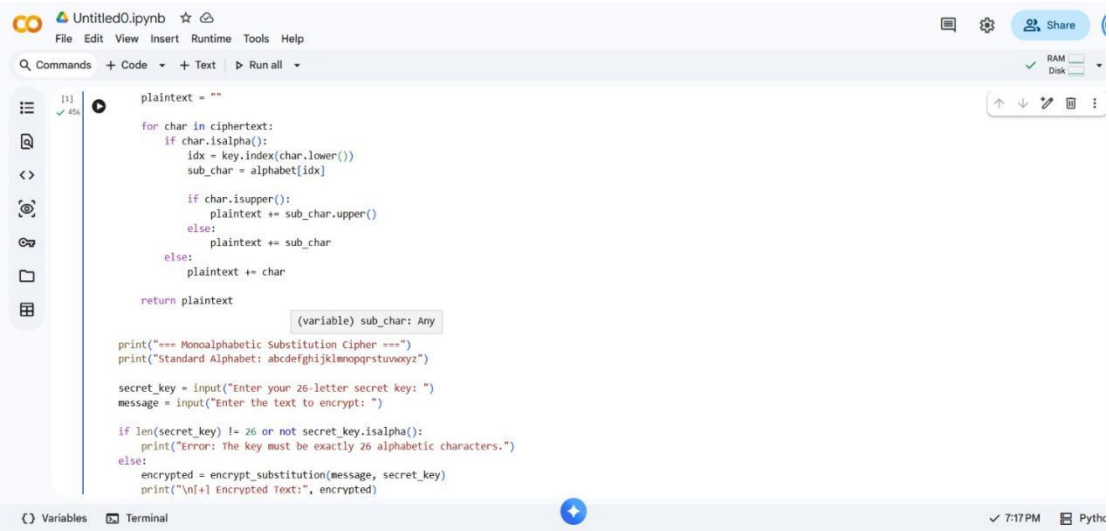
   c. If it is not a letter, keep it unchanged.

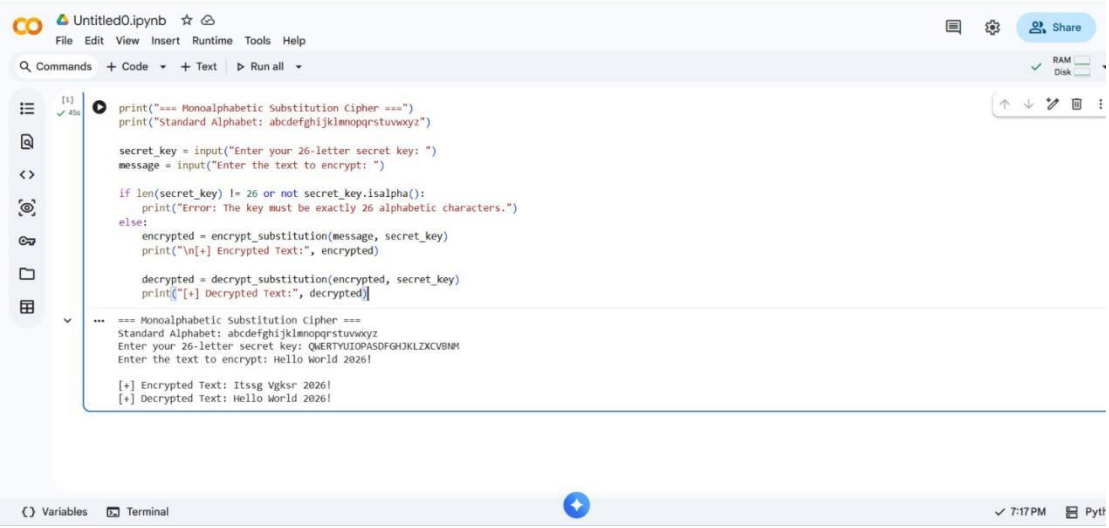
3. Display the recovered plaintext.

1. Monoalphabetic Substitution Cipher

- **Tool:** Python (Google Colab)
- **Key Used:** QWERTYUIOPASDFGHJKLZXCVBNM
- **Input:** Hello World 2026!
- **Purpose:** To implement encryption and decryption using a Monoalphabetic Substitution Cipher, where each plaintext letter is mapped to the corresponding letter in a scrambled key, and to reverse the mapping to recover the original message.
- **Observation:** The program successfully encrypted the input text "Hello World 2026!" into "Itssg Vgksr 2026!" using the given key, and correctly decrypted it back to the original plaintext, confirming that the substitution mapping is reversible.







Viva Questions

Q1. The Caesar cipher has only 25 possible keys, making it easy to brute-force. How many possible keys exist for a Monoalphabetic Substitution Cipher, and why does this make brute-forcing mathematically unfeasible?

Ans: A Monoalphabetic Substitution Cipher key is any permutation of the 26 letters of the alphabet, so the total number of possible keys is 26! = 403,291,461,126,605,635,584,000,000 (approximately 4 x 10^26). This is astronomically larger than Caesar's 25 keys. Even a computer testing one billion (10^9) keys per second would need roughly 10^10 years to try them all - longer than the age of the universe - making an exhaustive brute-force search mathematically unfeasible.

Q2. Even though brute-forcing is nearly impossible, substitution ciphers are easily broken using Frequency Analysis. Explain how an attacker would use this technique to decipher a long intercepted message.

Ans: Although the key space is huge, the substitution mapping stays fixed, so the statistical structure of the underlying language is preserved in the ciphertext. An attacker counts how often each letter appears and compares this distribution with known English letter frequencies (E, T, A, O, I, N being most common), guessing that the most frequent ciphertext letter maps to E, the next to T, and so on. Common digrams (TH, HE, IN, ER) and trigrams (THE, AND, ING), along with word-length clues, are also used to refine the guesses until the full key is reconstructed. The longer the message, the closer its letter frequencies match the expected distribution, making it easier to break.

Q3. In the Colab reference code, what is the purpose of the .isalpha(), .isupper(), and .lower() string methods during the encryption loop?

Ans: isalpha() checks whether a character is a letter, so only letters are substituted while spaces, digits and punctuation are copied unchanged. lower() converts a character to lowercase before finding its index, since the alphabet and key strings are defined in lowercase. isupper() checks whether the original character was uppercase, so the substituted output letter can be converted back to uppercase, preserving the original case in the ciphertext.